

Appunti di Algorithm Design

SGSS - Francesco Silvestri

FILIPPO GARGIULO

Ottobre - Dicembre 2025

Indice

1	Introduzione agli Algoritmi	2
2	Divide and Conquer	4
2.1	Sorting	4
2.2	Integer Multiplication	6
2.3	Problemi	8
3	Greedy	11
3.1	Dijkstra e Shortest Path	11
3.2	Priority Queue	14
3.3	Scheduling	14
4	Dynamic Programming	16
4.1	Bellman-Ford	16
4.2	Trovare i cicli negativi	18
4.3	Scheduling Pesato	19
5	NP-completezza	21
5.1	Decision Problems	23
5.2	Una gerarchia di difficoltà	24
6	Approximate and Randomized algorithms	28
6.1	Approximate Algorithms	28
6.1.1	Knapsack Problem	30
6.2	Randomized Algorithms	32
6.2.1	Quicksort	33
6.2.2	Min Cut	34
7	Direzioni di ricerca	37

§1 Introduzione agli Algoritmi

Esempio 1.1

Dati due interi positivi A e B con

$$A = \overline{a_{n-1}a_{n-2}a_{n-3} \dots a_0}$$

$$B = \overline{b_{n-1}b_{n-2}b_{n-3} \dots b_0}$$

Calcolare $C = A \cdot B$.

In questo esempio i due interi A e B sono l'input, e C è l'output. L'input ha sempre una **taglia**, ovvero un numero che caratterizza la sua grandezza. A voler essere rigorosi

Definizione 1.2 (Input Size). L'input size N è il numero di bit necessari a rappresentare l'input.

In questo caso l'input è costituito da $N = 2n$ cifre decimali. L'approccio naive di questo problema porta ad una complessità di $\mathcal{O}(n^2)$. Un miglioramento è già l'algoritmo di Karatsuba, che dà una complessità di circa $\mathcal{O}(n^{1.58})$.

Diamo la prima importante definizione

Definizione 1.3 (Computational Problem). Un problema computazionale Π è una relazione tra due insiemi I (insieme delle istanze) ed S (insieme delle soluzioni):

$$\Pi \subseteq I \times S$$

Nel caso dell'esempio 1 l'insieme delle istanze è l'insieme di tutte le possibili coppie ordinate di numeri interi ($I = \mathbb{N} \times \mathbb{N}$), mentre S è sempre l'insieme dei numeri naturali (tutti i possibili prodotti).

Definizione 1.4 (Algoritmo). Un algoritmo è una sequenza ben definita e finita di step base che trasforma un dato input ι in un unico output σ .

Il concetto di "step base" è praticamente primitivo, per esempio se lavoro con insiemi di numeri posso assumere che sommare e moltiplicare due cifre sia un'operazione base. Notiamo che in un problema computazionale prevede che una stessa istanza abbia più di una soluzione, mentre un algoritmo per come è definito qui è **una funzione**.

Definizione 1.5 (Risolvere un problema computazionale con un algoritmo). Diciamo che un algoritmo A risolve il problema computazionale $\Pi = I \times S$ se e solo se

1. $I \subseteq \text{dom}(f_A)$
2. $\forall i \in I, (\iota, f_A(\iota)) \in \Pi$

In realtà un algoritmo può avere un dominio anche più grande di I , e un codominio diverso da S (si pensi per esempio ai messaggi di errore che l'algoritmo può restituire).

Definizione 1.6 (encoding). Chiamiamo $\hat{I}$ il dominio dell'algoritmo f_A . Un encoding è una funzione $E : I \rightarrow \hat{I}$. Questa funzione spesso è elementare, ma per oggetti di I più complicati diventa più difficile da definire (si pensi ad esempio ai grafi, o alla rappresentazione dei numeri reali in virgola mobile).

Da qui in avanti chiameremo $\hat{I}$ il dominio dell'algoritmo f_A . Per progettare un algoritmo in modo che risolva un determinato problema computazionale dobbiamo verificare che l'algoritmo soddisfi le seguenti:

- 1) Correttezza $\rightarrow$ L'algoritmo A risolve effettivamente Π
- 2) Efficienza $\rightarrow$ l'algoritmo ha un running time ridotto

Definizione 1.7 (Running time). Sia A un algoritmo, allora

$$\hat{T}_A(\iota) = \# \text{operazioni da eseguire per } A(\iota)$$

è il *running time* dell'algoritmo per quello specifico input.

Definizione 1.8 (Worst Case Running Time). Definiamo $T_A(n)$ il *worst case running time* come

$$T_A(n) = \max_{\iota \in I, |\iota|=n} \{ \hat{T}_A(\iota) \}$$

dove n è l'input size di ι .

Il vantaggio dell'usare il worst case è che i casi peggiori sono di solito anche i più facili da trattare matematicamente. Per esempio abbiamo:

$$T_{\text{mergesort}}(n) \sim n \log(n)$$

$$T_{\text{bubblesort}}(n) \sim n^2$$

Definizione 1.9 (Notazione asintotica O grande). Date due funzioni $f(n)$ e $t(n)$ diciamo che:

$$t(n) = \mathcal{O}(f(n))$$

se esistono $c > 0, n_0 > 0$ tali che $t(n) \leq c \cdot f(n)$ per ogni $n \geq n_0$.

Per esempio

$$2n^2 + 3n - 1 = \mathcal{O}(n^2)$$

In questo corso useremo sempre il worst case running time per misurare l'efficienza di un algoritmo. Si tenga conto che questo non è l'unico modo, e che ce ne sono molti altri, più indicati in altri casi (per esempio quando si fa parallel computing).

Definizione 1.10 (Notazione omega grande). Per esprimere il tempo minimo che un algoritmo A può impiegare per risolvere il problema computazionale Π utilizziamo la notazione omega grande. Diciamo che

$$f(n) = \Omega(g(n))$$

se e solo se esistono due costanti $\varepsilon, n_0 > 0$ tali che

$$f(n) \geq \varepsilon g(n)$$

per ogni $n > n_0$.

Di solito non riusciamo davvero a trovare informazioni sul lower bound dell'algoritmo, e quando ci riusciamo non sono decisive. Questo eccetto alcuni esempi molto famosi.

§2 Divide and Conquer

La tecnica del divide and conquer consiste inizialmente nello scomporre un problema grande in problemi più piccoli (s_i). Questo ha senso quando i problemi più piccoli sono più "facili" di quelli grandi (divide). Il secondo passo è risolvere ogni sottoproblema s_i appena creato. Infine rimettiamo insieme tutti i sottoproblemi e risolviamo il problema grande (conquer).

§2.1 Sorting

Esempio 2.1

Si consideri il seguente problema computazionale:

Input Una sequenza di numeri interi $A = \{a_0, a_1, \dots, a_n\}$.

output Una permutazione di σ tale che $\sigma(A)$ sia in ordine crescente.

Un approccio è Bubble Sort, ovvero presa la lista itero sulle coppie adiacenti a partire dall'inizio, e le inverte se sono nell'ordine sbagliato. In questo modo alla fine della sequenza dopo ogni iterazione appare il numero più grande, poi il secondo più grande, poi il terzo più grande... questo dà una complessità di $\mathcal{O}(n^2)$.

Possiamo fare di meglio, infatti con Merge Sort otteniamo una complessità di $\mathcal{O}(n \log n)$. Merge Sort è una famosa applicazione di divide and conquer, e i due step ad alto livello seguono in questo modo:

Divide Dividiamo la sequenza di input in due sequenze lunghe la metà e ordiniamole.

Conquer Uniamo le due sequenze in tempo lineare in modo che il risultato sia ancora ordinato.

Algoritmo 2.2 (Merging) — Cominciamo con il dichiarare il problema computazionale

Input Due sequenze A e B ordinate.

Output Una sequenza $C = A \cup B$ ordinata.

Scriviamo lo pseudocodice:

```

1:  $n = |A| + |B|$ 
2:  $P_A \leftarrow 0$ 
3:  $P_B \leftarrow 0$ 
4:  $P_C \leftarrow 0$ 
5: vettore  $C(n)$ 
6:  $A.append(\infty)$ 
7:  $B.append(\infty)$ 
8: while  $P_C < n$  do
9:   if  $A[P_A] \leq B[P_B]$  then
10:     $C[P_C] \leftarrow A[P_A]$ 
11:     $P_C++$ 
12:     $P_A++$ 
13:   else
14:     $C[P_C] \leftarrow B[P_B]$ 
```

```

15:       $P_C ++$ 
16:       $P_B ++$ 
17:  end if
18: end while

```

Teorema 2.3 (Correttezza ed efficienza del merging)

L'algoritmo di Merging è corretto e richiede $T_{\text{merge}}(n) = \mathcal{O}(n)$.

Dimostrazione. Come detto prima, dobbiamo dimostrare due cose:

Correctness Procediamo per induzione, dimostriamo che alla fine dell' i -esima iterazione i primi i elementi di C sono i primi i elementi più piccoli di $A \cup B$.

Passo base $i = 0$: per la definizione del problema computazionale uno tra $A[0]$ e $B[0]$ è il minimo elemento di $A \cup B$, dunque il minore tra i due sarà proprio l'elemento cercato.

Supponiamo ora per induzione che i primi i elementi di C siano i più piccoli elementi di $A \cup B$ in ordine crescente. Per ipotesi del problema uno tra $A[P_A]$ e $B[P_B]$ conterrà il prossimo elemento più piccolo di $A \cup B$ (si dimostra per assurdo, se vuoi). Dunque se prendiamo $\min\{A[P_A], B[P_B]\}$ abbiamo esattamente quello che vogliamo, dato che C rimane ordinato e contiene i primi $i + 1$ elementi più piccoli di $A \cup B$.

Efficiency Osservando gli step dell'algoritmo notiamo che

$$T_{\text{merge}}(n) = \mathcal{O}(1) + \mathcal{O}(n) = \mathcal{O}(n)$$

In questo caso l'analisi è *stretta*, ovvero non esiste un'istanza che fa meno (asintoticamente) di $\mathcal{O}(n)$. In questo caso dunque si scrive

$$T_{\text{merge}}(n) = \Theta(n)$$

□

A questo punto usando questa primitiva possiamo effettivamente scrivere l'algoritmo del mergesort:

Algoritmo 2.4 (Mergesort) — Assumiamo l'implementazione precedente di Merge, allora

```

1: function MERGESORT( $A, n$ )
2:   if  $n \leq 1$  then
3:     return  $A$ 
4:   end if
5:   split  $A$  into  $A_1, A_2$  of size  $\lfloor n/2 \rfloor$  and  $\lceil n/2 \rceil$ .
6:    $A_1 \leftarrow \text{MERGESORT}(A_1, \lfloor n/2 \rfloor)$ 
7:    $A_2 \leftarrow \text{MERGESORT}(A_2, \lceil n/2 \rceil)$ 
8:    $\tilde{A} \leftarrow \text{MERGE}(A_1, A_2)$ 
9:   return  $\tilde{A}$ 
10: end function

```

A questo punto dimostriamone correttezza ed efficienza

Lemma 2.5 (Correttezza ed efficienza di Mergesort)

Mergesort ordina effettivamente l'input e va in $\mathcal{O}(n \log n)$.

Dimostrazione. La correttezza segue abbastanza chiaramente dalla correttezza del merge. Assumiamo che n sia una potenza di due, se così non è abbiamo delle costanti in più davanti al T , ma la complessità non cambierà. Consideriamo quindi l'albero delle chiamate di mergesort. Essenzialmente è un albero binario, dove ogni nodo è l'esecuzione di una chiamata dell'algoritmo. Esprimiamolo come

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + cn & n > 1 \\ c & n \leq 1 \end{cases}$$

Abbiamo messo la stessa costante perché abbiamo scelto la peggiore tra quelle di entrambi i casi. Ora se trovassimo una formula chiusa per questa successione avremmo finito. Tirando a indovinare che sia

$$T(n) = cn \log 2n$$

possiamo facilmente verificare che sia la funzione giusta e che soddisfi $T(n) = \mathcal{O}(n \log n)$. $\square$

§2.2 Integer Multiplication

A smart thing we can do to multiply better is to divide (and conquer).

[Sheafification of G \(Youtube\)](#)

Torniamo al problema della moltiplicazione di interi 1.1. Dalle scuole elementari sappiamo risolverlo in $\mathcal{O}(n^2)$. Sarebbe molto strano se potessimo fare di meglio... no? Non è ancora noto se sia possibile risolverlo in tempo lineare, tuttavia l'algoritmo di Karatsuba permette di ottenere una complessità di circa

$$T(n) = \mathcal{O}(n^{1.59}).$$

L'idea è applicare divide and conquer dividendo i due numeri in due sottonumeri lunghi la metà $A = (A_1, A_2)$ e $B = (B_1, B_2)$ (denotiamo con le parentesi la concatenazione tra numeri). A questo punto scrivendo

$$A = 10^{n/2} A_1 + A_2$$

$$B = 10^{n/2} B_1 + B_2$$

notiamo che la moltiplicazione è semplicemente

$$A \cdot B = 10^n A_1 B_1 + 10^{n/2} (A_1 B_2 + A_2 B_1) + A_2 B_2$$

e questo è interessante perché ci dà un modo per dividere il problema originale in sotto-problemi più piccoli. Abbiamo in questo modo giustificato il seguente algoritmo:

Algoritmo 2.6 (Integer Multiplication) — Siano A e B due numeri come descritti in 1.1. Allora

```

1: function INTEGERMULT( $A, B$ )
2:   if  $n = 1$  then
3:     return  $A \cdot B$ 
4:   end if
```

```

5:   split  $A$  into  $A_1, A_2$  of size  $n/2$ 
6:   split  $B$  into  $B_1, B_2$  of size  $n/2$ 
7:    $C_4 \leftarrow \text{INTEGERMULT}(A_2, B_2)$ 
8:    $C_3 \leftarrow \text{INTEGERMULT}(A_2, B_1)$ 
9:    $C_2 \leftarrow \text{INTEGERMULT}(A_1, B_2)$ 
10:   $C_1 \leftarrow \text{INTEGERMULT}(A_1, B_1)$ 
11:   $C \leftarrow (C_4, C_3 + C_2, C_1)$     //concatenazione
12: end function

```

Facciamo l'analisi e... troviamo che la complessità è sempre $\mathcal{O}(n^2)$. Il problema è che il modo in cui abbiamo diviso ci porta a fare un numero di operazioni più che lineare (il costo di tutti i sottoproblemi è $2n$). Per applicare un divide and conquer utile dobbiamo ricorrere in modo da avere sottoproblemi il cui running time complessivo sia minore del problema originale.

Si arriva all'algoritmo di Karatsuba notando che in realtà i numeri che dobbiamo concatenare sono solo 3, ovvero C_1 , C_4 e $C_3 + C_2$. In particolare facciamo questo calcolando il prodotto

$$(A_1 + A_2)(B_1 + B_2) = A_1B_1 + A_1B_2 + A_2B_1 + A_2B_2$$

e notando che otteniamo $C_3 + C_2$ sottraendo ad esso $C_1 + C_4$. Dunque implementandolo abbiamo:

Algoritmo 2.7 (Karatsuba Multiplication) — Siano A e B due numeri come descritti in 1.1. Allora

```

1: function KARATSUBAMULT( $A, B$ )
2:   if  $n = 1$  then
3:     return  $A \cdot B$ 
4:   end if
5:   split  $A$  into  $A_1, A_2$  of size  $n/2$ 
6:   split  $B$  into  $B_1, B_2$  of size  $n/2$ 
7:    $\tilde{A} \leftarrow A_1 + A_2$ 
8:    $\tilde{B} \leftarrow B_1 + B_2$ 
9:    $C_4 \leftarrow \text{KARATSUBAMULT}(A_2, B_2)$ 
10:   $C_0 \leftarrow \text{KARATSUBAMULT}(\tilde{A}, \tilde{B})$ 
11:   $C_1 \leftarrow \text{KARATSUBAMULT}(A_1, B_1)$ 
12:   $K \leftarrow C_0 - C_1 - C_4$ 
13:   $C \leftarrow (C_4, K, C_1)$     //concatenazione
14: end function

```

Anche qui facendo l'analisi come nei casi precedenti troviamo

$$T(n) = \begin{cases} c(3T(\frac{n}{2}) + 3n) & n > 1 \\ c & n \leq 1 \end{cases}$$

che svolgendo per induzione e/o provando a tirare a indovinare la formula chiusa ci dà una complessità di

$$T(n) = \mathcal{O}(n^{\log_2 3})$$

abbastanza sorprendente non c'è che dire.

§2.3 Problemi

Problema 2.8. Dati n punti in $\mathbb{R}^2$ trovare la coppia di punti con distanza minore. In altre parole siano $A \subset \mathbb{R}^2$ con $|A| = n$, trovare

$$(p_1, p_2) = \operatorname{argmin}_{p, q \in A} \{d(p, q), p \neq q\}$$

Soluzione. Dividiamo i punti a metà "tirando una riga" tale che i due semipiani generati da essa contengano $\lfloor \frac{n}{2} \rfloor$ e $\lceil \frac{n}{2} \rceil$. Cerchiamo per ogni metà i punti che minimizzano la distanza ricorrendo. Ora prendiamo il minimo delle due distanze trovate. Questo ci dà informazioni su un'eventuale coppia "a cavallo" della linea di separazione che minimizza la distanza.

Ora dobbiamo fare il merge... come?

Tracciamo due nuove linee parallele a distanza δ dalla linea originale, con δ la distanza minima tra i punti trovati prima.

Dividiamo la regione di piano tra le due nuove rette in quattro colonne di quadratini di lato $\delta/2$. Questo riduce notevolmente la quantità di comparazioni che dobbiamo fare, perché sicuramente sappiamo che se c'è una coppia con distanza minore di δ sarà a cavallo della linea, dunque in ogni quadratino non ci può essere più di un punto. Inoltre ogni quadratino dobbiamo confrontarlo solo con un numero finito e costante di quadratini (dall'altro lato della retta).

Dunque questo si può fare in tempo lineare (l'implementazione è un altro paio di maniche). Ma si può fare. Vediamo come

Algoritmo 2.9 (Soluzione) — Supponiamo di avere una primitiva $D(p_1, p_2)$ che permette di calcolare la distanza tra due punti in $\mathcal{O}(1)$.

```

1: function CLOSESTPAIR( $P$ ) //  $P$  insieme di  $n$  punti in  $\mathbb{R}^2$ 
2:    $P_x \leftarrow \text{sort } P \text{ on } x$ 
3:    $P_y \leftarrow \text{sort } P \text{ on } y$ 
4:   return CLOSESTPAIRR( $P_x, P_y$ )
5: end function
6: function CLOSESTPAIRR( $P_x, P_y$ )
7:   if  $|P_x| \leq 3$  then:
8:     Find closest pair  $(p_1, p_2)$  with brute force
9:     return  $(p_1, p_2)$ 
10:  end if
11:  Split  $P_x$  in two halves of size  $n/2$ :
12:  Construct  $q_x, q_y, r_x, r_y$ 
13:   $q_x \leftarrow$  subseq of points of  $p_x$  with  $x$  coordinate  $L \leq x$ 
14:   $q_y \leftarrow$  subseq of points of  $p_y$  with  $x$  coordinate  $L \leq x$ 
15:   $r_x \leftarrow$  subseq of points of  $p_x$  with  $x$  coordinate  $L > x$ 
16:   $r_y \leftarrow$  subseq of points of  $p_y$  with  $x$  coordinate  $L > x$ 
17:   $(q_0, q_1) \leftarrow \text{CLOSESTPAIRR}(q_x, q_y)$ 
18:   $(r_0, r_1) \leftarrow \text{CLOSESTPAIRR}(r_x, r_y)$ 
19:   $\delta \leftarrow \min(d(q_0, q_1), (r_0, r_1))$ 
20:   $S =$  points in  $p_x$  with distance  $< \delta$  from  $L$  construct
21:  Construct  $S_y$  from  $p_y$ .
22:  for all  $z \in S_y$  do
23:    Compare the distance of  $z$  with the
24:  end for
25: end function
```

Facendo un'analisi temporale troviamo che

$$T(n) = \begin{cases} 2T(n/2) + \mathcal{O}(n) & n > 3 \\ \mathcal{O}(1) & n \leq 3 \end{cases}$$

che da una complessità di $\mathcal{O}(n \log n)$.

A meno di implementazione dunque abbiamo finito. $\square$

§3 Greedy

Per parlare di algoritmi greedy ci concentreremo su problemi di grafi. Un algoritmo Greedy fa sempre la scelta ottima in un certo istante.

Esempio 3.1 (Knapsack problem)

Uno zaino ha volume 8. Vogliamo riempirlo con pezzi di volume 2 o 7 in modo da massimizzare il numero di pezzi. Un algoritmo greedy ragionerebbe per esempio prendendo sempre i pezzi più grandi, prendendo decisioni non per forza ottime a lungo termine.

Ripassiamo un attimo la definizione di grafo

Definizione 3.2 (Grafici e affini). Un grafo G è una coppia di insiemi V ed E . V è un insieme di n nodi (o vertici), mentre $E \subseteq V \times V$ è un insieme di m archi che connettono coppie di nodi.

Un grafo si dice *diretto* se l'insieme degli archi è formato da coppie ordinate, *indiretto* se l'insieme degli archi è formato da insiemi disordinati.

Gli archi possono essere *cappi*, ovvero coppie del tipo $\{v, v\}$, dunque gli elementi di E sono multiset.

Un grafo può essere *pesato*, ovvero ad ogni arco in E si può associare un *peso* ovvero un numero che codifica una certa informazione.

Definizione 3.3 (Vicinati). Di un vertice $v \in V$ di un grafo si può definire il vicinato, ovvero l'insieme degli elementi di V che condividono un arco con v .

Definizione 3.4 (Percorso). Un percorso su un grafo G dal nodo a al nodo b è una sequenza di archi consecutivi (ovvero che condividono un nodo con quello che viene subito dopo) che "va da a a b ".

In modo forse più formale ed equivalente, un percorso è una sequenza di vertici

$$a, v_1, v_2, \dots, b$$

tale che esiste un arco da ogni nodo a quello successivo (se ce n'è più di uno dobbiamo scegliere in ogni caso quali archi prendere per avere un vero percorso).

Definizione 3.5 (Costo di un percorso). Se un grafo è pesato, il costo di un percorso è la somma dei suoi archi. Altrimenti è semplicemente il numero di archi del percorso.

Trattando di algoritmi su grafi, ci tornerà utile esprimere le complessità in funzione del numero di archi. Questo perché solitamente i grafi sono *sparsi*:

Definizione 3.6 (Grafo sparso). Un grafo G con n nodi m archi si dice sparso se e solo se $m \ll n$.

Vediamo ora un modo di applicare l'approccio greedy ai grafi.

§3.1 Dijkstra e Shortest Path

Esempio 3.7 (Single Source Shortest Path/SSSP)

Definiamo il seguente problema computazionale:

Input Un grafo diretto pesato connesso (non necessariamente fortemente connesso) con n vertici ed m archi di pesi non negativi. Un vertice particolare del grafo denotato con s . Si assume che s sia connesso con tutti gli altri nodi (ovvero che per ogni nodo ci sia un percorso che parte da s e va in quel nodo).

Output Una lista di numeri $x_1, x_2, \dots, x_{n-1}$ rappresentanti il minimo costo di un percorso che parte da s e si dirige in ciascuno degli altri $n - 1$ nodi.

Da qui in poi denotiamo con $d(s, u)$ il costo del percorso di costo minimo da s a u . Dato che il grafo è diretto purtroppo è possibile che

$$d(s, u) \neq d(u, s)$$

Esempio 3.8 (All Pairs Shortest Path / APSP)

è analogo ma richiede di calcolare le distanze tra ogni coppia di nodi.

Per risolvere SSSP esiste un algoritmo greedy inventato nel 1959 da Dijkstra. La principale intuizione consiste nel ricorrere su sottoproblemi già risolti. Sia V' un insieme di nodi per cui abbiamo già calcolato gli shortest path e V l'insieme di tutti i vertici eccetto s . Il senso di Dijkstra è che possiamo boundare il costo minimo del seguente arco quindi possiamo stimare

$$d(s, u) = \min_{e=(a,u), a \in V'} \{d(s, a) + w_e\}$$

e quindi

$$V' \leftarrow V' \cup \left\{ \underset{u \in V \setminus V'}{\operatorname{argmin}} \{d(s, u)\} \right\}$$

vediamo lo pseudocodice quindi

Algoritmo 3.9 (Dijkstra $\mathcal{O}(nm)$) — Assumiamo le notazioni dell'esempio sopra, allora:

```

1: function DIJKSTRA( $G : (V, E)$ ,  $s \in V$ )
2:    $V' := \{s\}$  //l'insieme dei nodi di cui abbiamo già trovato lo shortest path
3:    $d$  array of numbers //conterrà le distanze minime che stiamo cercando
4:   for all  $v \in V$  do
5:      $d[v] \leftarrow 0$  if  $v = s$ 
6:      $d[v] \leftarrow w_{(s,v)}$  if  $(s, v) \in E$ 
7:      $d[v] \leftarrow \infty$  otherwise.
8:   end for
9:   while  $|V'| < n$  do
10:     $d_{min} := \infty$ 
11:     $v_{min} := \text{null}$ 
12:    for all  $u \in V \setminus V'$  do
13:      if  $d[u] < d_{min}$  then
14:         $d_{min} \leftarrow d[u]$ 
15:         $v_{min} \leftarrow u$ 

```

```

16:         end if
17:     end for
18:      $V' \leftarrow V' \cup \{v_{min}\}$ 
19:     for all  $(v_{min}, u) = e \in V \setminus V'$  do
20:          $d[u] \leftarrow \min \{d[u], d[v_{min}] + w_e\}$ 
21:     end for
22: end while
23: return  $d$ 
24: end function

```

Lemma 3.10 (Dijkstra)

L'algoritmo di Dijkstra è corretto e impiega $T(n, m) = \mathcal{O}(nm)$ operazioni.

Dimostrazione. Dobbiamo come sempre dimostrare due cose diverse:

Correttezza Procediamo per induzione sulla grandezza di V' . Il caso base è $|V'| = 1$ che è triviale in quanto non ci sono archi con peso negativo (e quindi cappi negativi). Supponiamo per induzione allora che l'algoritmo sia corretto per $|V'| \leq k$, dimostriamo che è corretto anche per $|V'| = k + 1$. Per ipotesi induttiva conosciamo lo shortest path di tutti i k nodi in V' . Dimostriamo che in effetti possiamo aggiungere un nodo a V' , chiamiamolo $\hat{u} \in V \setminus V'$, tale che $d[\hat{u}]$ è il minimo tra tutti gli elementi di $V \setminus V'$. Supponiamo per assurdo che vi sia un cammino da V' a $V \setminus V'$ con un costo maggiore di $d[\hat{u}]$, affinché ciò sia possibile il percorso deve avere almeno un nodo fuori da V' , infatti se così non fosse nel primo ciclo for avremmo considerato quel percorso e avremmo aggiornato d di conseguenza. Sia y questo nodo. Per come abbiamo scelto $\hat{u}$ sappiamo che

$$d[\hat{u}] \leq d[y]$$

ma dato che stiamo supponendo che un percorso per y sia minore di $d[\hat{u}]$ abbiamo

$$d[\hat{u}] > d[y] + d(y, \hat{u})$$

e concatenando e usando il fatto che tutti i percorsi pesano al minimo 0 (non ci sono archi negativi) abbiamo:

$$d[y] \geq d[\hat{u}] > d[y] + d(y, \hat{u}) > d[y]$$

assurdo. Dunque $d[\hat{u}]$ è effettivamente il peso dello shortest path, e quindi possiamo aggiungere $\hat{u}$ a V' . Questo chiude per induzione.

Efficienza Vediamo che l'algoritmo svolge un ciclo while in n iterazioni, e ad ogni iterazione fa un numero lineare di operazioni in m , dunque abbiamo una complessità di $\mathcal{O}(nm)$.

Dunque abbiamo dimostrato sia efficienza che correttezza, e abbiamo finito. $\square$

Il collo di bottiglia del programma precedente è l'iterazione su *tutti* gli archi quando dobbiamo aggiungere un nodo V' . In effetti sembra strano che sia necessario per forza controllare tutti quegli archi, dovrebbe esserci un modo per svolgere quello step in modo più efficiente. Questo è ottenuto tramite una priority queue.

§3.2 Priority Queue

Una priority queue è un esempio di **struttura dati**, ovvero un insieme di dati a cui sono associati alcuni algoritmi che prendono in input alcuni dei valori memorizzati nella struttura.

In particolare una priority queue è una "scatola nera" che ha implementate tre operazioni:

- **Init**: inizializza la struttura e riceve in input n coppie (chiave, valore) da mettere nella struttura.
- **Insert**: fa quello che ti aspetti.
- **Give minimum**: ritorna la coppia con il valore minimo.
- **Change value(k, n)**: inserisce una coppia con una determinata chiave, k , se non esiste già una coppia con la stessa chiave e valore. Se esiste già una coppia con quella chiave e con quel valore, inserisce una coppia con la stessa chiave ma con diverso valore.

Ci sono vari modi di implementare questa struttura, quello che probabilmente dà i tempi migliori (con un binary heap ad esempio) permette di avere init in $\mathcal{O}(n \log n)$, e tutte le altre operazioni in $\mathcal{O}(\log n)$ (ammortizzato). Usando questa struttura quindi possiamo implementare dijkstra $\mathcal{O}(n \log m)$.

§3.3 Scheduling

Esempio 3.11

Supponiamo di avere n task da svolgere, ognuno con un suo tempo di start $s_i \in \mathbb{R}^+$ e un suo tempo di finish $s_i < f_i \in \mathbb{R}^+$. Supponiamo anche di poter eseguire solo un task alla volta, di non poter interrompere un task e di poter fare il task solo iniziandolo a s_i e finendolo a f_i . Qual è il massimo numero di task che posso eseguire? Più formalmente

Input Un insieme R di n coppie $(s_i, f_i) \in \mathbb{R}_{\geq 0}^2$ tali che per ogni i $s_i < f_i$.

Output Un insieme massimale $S \subseteq R$ di task compatibili, ovvero un insieme tale che ogni coppia di task è compatibile. Due task (s_i, f_i) e s_j, f_j si dicono compatibili se e solo se $s_i < f_i < s_j < f_j$ oppure $s_j < f_j < s_i < f_i$.

Un primo approccio greedy a questo problema sarebbe semplicemente quello di scegliere il task con il tempo s_i minimo non già scelto. E questo in effetti funziona, ma solo se tutti gli eventi hanno la stessa lunghezza (ovvero se $f_i - s_i = f_j - s_j$ per ogni i, j), tuttavia non è difficile inventarsi un controesempio se le lunghezze degli intervalli sono diverse. Proviamo allora a scegliere sempre le task con durata minima. Anche questo fallisce, per esempio quando un intervallo molto piccolo si accavalla con due intervalli grandi consecutivi.

Un altro approccio ancora sarebbe quello di contare per ogni task k il numero di task in conflitto con esso $c(k)$ e scegliere sempre il minimo tra questi. Anche questo purtroppo non funziona per controesempi con tante cose messe a cascata.

L'idea corretta in realtà è quella di scegliere sempre l'evento che finisce prima (o quello con il massimo tempo di start equivalentemente).

Algoritmo 3.12 (Scheduling Greedy) — Assumiamo la notazione introdotta prima:

```

1: function SCHEDULING( $R$ )
2:   SORT( $R$ , less ending time)
3:    $S := \emptyset$ 
4:   for  $j = 0$  to  $n - 1$  do
5:     if  $R[j]$  is compatible with  $S$  then
6:        $S \leftarrow S \cup \{R[j]\}$ 
7:     end if
8:   end for
9:   return  $S$ 
10: end function

```

La complessità è $\mathcal{O}(n \log n)$ se per ogni iterazione del for svolgiamo $\mathcal{O}(1)$ operazioni, ma questo è chiaramente sempre possibile dato che S è ordinato, e quindi ci basta confrontare $R[j]$ con l'elemento più grande di S . Ci manca

Lemma 3.13 (Correctness dello Scheduling Greedy)

Il programma dello scheduling greedy restituisce un insieme che ha la cardinalità di una soluzione massimale.

Dimostrazione. Siano $i_0, i_1, \dots, i_{k-1}$ gli elementi di S ordinati per tempo di fine. Supponiamo di avere un "oracolo" che ci dia una soluzione effettivamente massimale A composta dagli elementi $j_0, j_1, \dots, j_{h-1}$.

Supponiamo per assurdo che $h > k$. Dimostriamo per induzione che " S è sempre in vantaggio su A " ovvero

$$f_{i_l} \leq f_{j_l}.$$

Il caso base è banale, perché alla prima iterazione prendiamo semplicemente il minimo di R . Per ipotesi induttiva ora abbiamo

$$f_{i_{l-1}} \leq f_{j_{l-1}}$$

ora, chiaramente dato che A è compatibile abbiamo

$$f_{j_{l-1}} < s_{j_l}$$

ma quindi per transitività

$$f_{i_{l-1}} < s_{j_l}$$

ma questo significa che j_l era compatibile con $i_0, \dots, i_{l-1}$ e dunque dato che f_{i_l} è il minimo nell'insieme delle task compatibili abbiamo

$$f_{i_l} \leq f_{j_l}$$

come richiesto. Ora, A ha almeno un task in più rispetto ad S , ovvero $j_k \in A$ ma $j_k \notin S$, però j_k è in effetti compatibile con S , perché sappiamo che

$$f_{i_{k-1}} \leq f_{j_{k-1}}$$

per quanto dimostrato prima, e inoltre

$$s_{j_k} \geq f_{j_{k-1}}$$

dunque

$$s_{j_k} \geq f_{i_{k-1}}$$

ma allora S deve contenere anche j_k , il che è assurdo perché implicherebbe $|S| = k+1$. $\square$

§4 Dynamic Programming

La programmazione dinamica (nome inventato da Bellman per attrarre fondi di ricerca) consiste come nel divide and conquer nello scomporre un problema principale in sottoproblemi. La differenza principale è che i sottoproblemi **non sono disgiunti**. Dunque per calcolare la soluzione all'unione di più sottoproblemi bisogna sforzarsi un po' di più e di solito implementare effettivamente una procedura specializzata per farlo.

In un certo senso un programma DP fa una sorta di bruteforce in cui itera su tutti i possibili sottoproblemi in modo intelligente, per poi unificarli in modo efficiente.

In un altro senso ancora un programma DP di solito fa un'*induzione estesa* sul problema.

§4.1 Bellman-Ford

Dijkstra faceva una grossa assunzione, ovvero che non ci fossero archi di peso negativo. In realtà questa condizione è anche troppo stringente per trovare un SSSP, infatti esistono grafi con pesi negativi per cui è sempre possibile trovare un SSSP. Il problema vero si ha quando ci sono dei *cicli* negativi, in quel caso infatti entrando in un ciclo e percorrendolo un numero arbitrario di volte possiamo rendere il costo di ogni percorso arbitrariamente basso. Supponiamo allora d'ora in poi che non ci siano cicli negativi. Utilizziamo l'algoritmo di Bellman-Ford per generalizzare quindi Dijkstra (è un algoritmo scoperto quattro volte, nel 1955 da Chimbél, nel 1958 da Bellman, nel 1956 da Ford, nel 1959 da Moore).

L'idea principale di Bellman-Ford (nel senso della DP) è assumere di conoscere già tutti i cammini minimi con al più $i \geq 0$ archi. Ovvero per ogni nodo v assumiamo di aver calcolato il percorso che va da s a v utilizzando al più i archi e che tra tutti i percorsi che usano al più i archi abbia costo minimo. Questo è un vincolo chiaramente, non ci permette di calcolare direttamente la soluzione. Tuttavia notiamo che per $i \rightarrow \infty$ le soluzioni diventano sempre più vicine alle ottimali.

L'idea è aumentare i gradualmente. Notiamo che in realtà per la natura degli shortest path un insieme di percorsi minimali che partono da s e vanno a tutti gli altri nodi costituiscono uno spanning tree del grafo. Dunque non ci possono essere più di $n - 1$ archi, e un percorso minimo dunque non può avere più di $n - 1$ archi. Dunque aumentando i dovremmo fare al massimo $n - 1$ iterazioni.

Manca il caso base, che è quando $i = 0$. In quel caso la distanza dell'origine è 0, mentre le distanze di tutti gli altri nodi sono ∞ .

Vediamo di definire una pseudoimplementazione: $\text{opt}(i, v)$: costo del percorso minimo con $\leq i$ archi da s a v . Chiaramente $\text{opt}(0, v) = 0$ se $v = s$, e $\text{opt}(0, v) = \infty$ altrimenti. Sia P l'arco che effettua quanto detto da opt . Dividiamo in due casi

P ha strettamente meno di i archi In questo caso semplicemente abbiamo

$$\text{opt}(i, v) = \text{opt}(i - 1, v)$$

P ha esattamente i archi In questo caso possiamo decomporre P in due percorsi, P' (lungo $i - 1$) e un singolo arco dal nodo u al nodo v . Notiamo che P' deve essere ottimale, quindi possiamo scrivere

$$\text{opt}(i, v) = \text{opt}(i - 1, u) + w_{u,v}$$

Dunque il cuore della DP è proprio questo. Possiamo scrivere

$$\text{opt}(i, v) = \min \left\{ \text{opt}(i - 1, v), \min_{n \in V \setminus \{v\}} \{ \text{opt}(i - 1, u) + w_{u,v} \} \right\}$$

che assumendo non ci siano cappi non nulli (si vede facilmente che questo non rompe niente) possiamo riscrivere come

$$\text{opt}(i, v) = \min_{u \in V} \{\text{opt}(i-1, u) + w_{v,u}\}$$

come accade molto spesso nelle DP possiamo schematizzare tutto con una matrice, ovvero alla riga i -esima memorizziamo il valore di $\text{opt}(i, v)$ al variare di tutti i v .

$$\begin{bmatrix} +\infty & +\infty & +\infty & 0 & +\infty & \infty \\ & & \vdots & & & \\ \text{opt}(n-1, v_1) & \text{opt}(n-1, v_2) & \text{opt}(n-1, v_3) & \text{opt}(n-1, s) & \text{opt}(n-1, v_4) & \text{opt}(n-1, v_5) \end{bmatrix}$$

ogni cella della matrice viene riempita iterando sulla riga sopra e calcolando il minimo. Dunque alla fine, nell'ultima riga, avremo il risultato che cerchiamo.

Algoritmo 4.1 (Bellman-Ford) — Assumiamo di non avere cicli negativi, allora

```

1: function BL( $G, s$ )
2:    $M := n \times n$  matrix
3:    $M[0, v] \leftarrow 0$  if  $s = v$ ,  $\infty$  otherwise
4:   for  $i = 1$  to  $n - 1$  do
5:     for all  $v \in V$  do
6:        $M[i, v] = \min_{u \in V} \{M[i-1, u] + w_{v,u}\}$ 
7:     end for
8:   end for
9:   return last row of  $M$ .
10: end function
```

Scritto così l'algoritmo è **molto** inefficiente, in effetti alloca una matrice $n \times n$ spendendo $\mathcal{O}(n^2)$ memoria e facendo $\mathcal{O}(n^3)$ operazioni. In effetti però il costo non dipende in alcun modo da m .

La prima ottimizzazione che possiamo fare è iterare ogni volta che calcoliamo il min solo sulla lista di adiacenza (inversa in realtà, dato che il grafo è diretto) del nodo v . Ma in questo modo ogni arco viene utilizzato esattamente una volta, dunque nel for all faremo al più m operazioni, il che porta la complessità a $\mathcal{O}(nm)$.

La seconda ottimizzazione riguarda la memoria. Non ci serve davvero memorizzare tutta la matrice, ci basta una (al massimo due) righe alla volta. Dunque la memoria passa a $\mathcal{O}(n)$.

E se volessimo avere un modo per costruire esplicitamente tutti i percorsi minimi? Per fare questo dobbiamo introdurre un'altra matrice:

$\text{prev}[i, v]$ = il nodo che dava il minimo (eccetto v) valore quando abbiamo calcolato $M[i, v]$

oppure più formalmente

$$\text{prev}[i, v] = \begin{cases} \arg \min_{u \in V} \{M[i-1, u] + w_{v,u}\} & \arg \min_{u \in V} \{M[i-1, u] + w_{v,u}\} \neq v \\ \text{prev}[i-1, v] & \arg \min_{u \in V} \{M[i-1, u] + w_{v,u}\} = v \end{cases}$$

In questo modo l'ultima riga di **prev** delinea l'*albero* delle distanze minime. Per ricavare il percorso minimo possiamo semplicemente fare

Algoritmo 4.2 (DFS) — Assumiamo di aver generato `prev` come descritto prima, consideriamolo qui semplicemente come un array (l'ultima riga della matrice):

```

1: function SHORTESTPATH( $v$ )
2:    $P = (v)$  the path starting from  $v$ .
3:   while  $\text{prev}[v] \neq v$  do
4:      $v \leftarrow \text{prev}[v]$ 
5:      $P \leftarrow (P, v)$ 
6:   end while
7:   Reverse  $P$ 
8:   return  $P$ 
9: end function

```

§4.2 Trovare i cicli negativi

Per trovare lo shortest path abbiamo ipotizzato non ci fossero cicli negativi, tuttavia, come capiamo se un grafo generico verifica questa ipotesi?

Un bruteforce non è davvero un'opzione (dovremmo verificare tutti i loop, abbastanza eccessivo). Per fortuna l'algoritmo di Bellman-Ford alla n -esima iterazione ci permette di verificare se esistono cicli negativi, vediamo come:

Prima di tutto eseguiamo BF come al solito per le prime $n - 1$ iterazioni, generando `opt[i, v]` come prima. A questo punto facciamo un'altra iterazione (fare questo non aumenta la complessità, come è evidente), abbiamo due casi:

- Se non esistono loop negativi, la prima riga rimane la stessa.
- Se esiste un loop negativo la prima riga diminuirà.

Dimostriamolo:

Lemma 4.3 (Trovare i cicli negativi)

L'algoritmo di bellman-ford ci permette di rilevare cicli negativi all' n -esima iterazione.

Dimostrazione. Supponiamo che esista un ciclo $v_0, \dots, v_k$ tale che

$$\sum_{i=0}^{k-1} w_{v_i, v_{i+1}} < 0$$

assumiamo per assurdo che `opt[n, v] = opt[n - 1, v]` per tutti i v . Questo vuol dire che

$$\text{opt}[n - 1, u] + w_{vu} \geq \text{opt}[n, v]$$

per ogni v e per ogni u . A questo punto quindi sommando la disuguaglianza sopra su tutti i v :

$$\sum_{i=0}^{k-1} \text{opt}[n - 1, v_i] \leq \sum_{i=0}^{k-1} \text{opt}[n, u_i] + w_{u_i v_i}$$

possiamo stimare dall'alto questa somma con $u_1, u_2, \dots, u_k$ i vertici tali che

$$u_i = v_{i-1}$$

allora abbiamo

$$\sum_{i=0}^{k-1} \text{opt}[n - 1, v_i] \leq \sum_{i=0}^{k-1} \text{opt}[n, v_{i-1}] + w_{v_{i-1} v_i}$$

a questo punto dunque "ruotando" gli indici arriviamo a

$$\dots \leq \sum_{i=0}^{k-1} \text{opt}[n, v_{i-1}] + w_{v_{i-1}v_i} = \sum_{i=0}^{k-1} \text{opt}[n, v_i] + \sum_{i=0}^{k-1} w_{v_{i-1}v_i}$$

ma quindi

$$\sum_{i=0}^{k-1} \text{opt}[n, v_i] \leq \sum_{i=0}^{k-1} \text{opt}[n, v_i] + \sum_{i=0}^{k-1} w_{v_{i-1}v_i}$$

ovvero

$$0 \leq \sum_{i=0}^{k-1} w_{v_{i-1}v_i}$$

assurdo. D'altro canto se non esistono loop negativi, abbiamo dimostrato prima che Bellman Ford trova lo shortest path, dunque essendo il minimo di un insieme sempre *unico*, la prima riga dovrà rimanere uguale. Da cui la tesi. $\square$

§4.3 Scheduling Pesato

Ritorniamo al problema dello scheduling:

Esempio 4.4

Assumiamo la definizione di insieme compatibile come sopra:

Input Un insieme R di n triplette $(s_i, f_i, w_i) \in \mathbb{R}_{\geq 0}^3$ tali che per ogni i $s_i < f_i$.

Output Un insieme $S \subseteq R$ di task compatibili tale che la somma dei valori w_i di tutti gli elementi dell'insieme è massima.

La strategia ottimale in questo caso è fare una DP. Come spesso accade per le DP, qui bisogna fare un ragionamento di "metti o toglì", ovvero partiamo dall'inizio e per ogni evento consideriamo due opzioni "metti" e "togli" e per ogni opzione calcoliamo il valore massimo. Cerchiamo di essere un po' più analitici: definiamo un vettore $P[i]$ che contiene per ogni task v_i il più grande indice j tale che v_i e v_j sono compatibili e $f_{v_i} > f_{v_j}$. Generare i $P[i]$ è facile ordinando gli eventi per tempo di fine e facendo una binary search. Dunque abbiamo una complessità di $\mathcal{O}(n \log n)$.

Se il task i sta nella soluzione ottima possiamo ignorare tutti i task a partire da $P[i]$ fino a $i - 1$. Definiamo dunque $\text{opt}[i]$ il costo della soluzione ottima usando i task fino a i incluso. Le possibilità dunque sono due

$$\text{opt}[i] = \begin{cases} \text{opt}[i-1] & \text{se } i \text{ non appartiene alla soluzione ottima} \\ \text{opt}[P[i]] + w_i & \text{se } i \text{ appartiene alla soluzione ottima} \end{cases}$$

incredibilmente, questo ci delinea proprio una DP! in particolare una cosa del tipo

$$\text{opt}[i] = \max \{ \text{opt}[i-1], \text{opt}[P[i]] + w_i \}$$

l'unica cosa che manca sono i casi base, ma si vede facilmente che

$$\text{opt}[-1] = 0$$

Benissimo, scriviamo lo pseudocodice

Algoritmo 4.5 (Scheduling Pesato Iterativo) — Assumiamo come prima tutte le notazioni dell'esempio.

```
1: function COMPUTE-OPT( $R$ )
2:    $P :=$  vector of length  $n$ .
3:    $opt :=$  vector of length  $n + 1$ .
4:    $opt[-1] = 0$ 
5:   COMPUTE-P( $R$ )
6:   for  $i = 0$  to  $n - 1$  do
7:      $opt[i] = \max \{opt[i - 1], opt[P[i]] + w_i\}$ 
8:   end for
9: end function
```

Algoritmo 4.6 (Scheduling Pesato Ricorsivo) — Assumiamo come prima tutte le notazioni dell'esempio. Assumiamo di avere già P generato.

```
function COMPUTE-OPT( $i, R$ )
  if  $i = -1$  then return 0
  end if
  return  $\max(\text{COMPUTE-OPT}(i - 1, R), \text{COMPUTE-OPT}(P[i], R))$ 
end function
```

Qual è il costo di questi algoritmi? Quello iterativo è abbastanza chiaramente lineare, quindi abbiamo una complessità di $\mathcal{O}(n \log n)$. Quello ricorsivo invece per ogni chiamata a funzione chiama la funzione stessa altre due volte, finendo per avere un complessità $\mathcal{O}(2^n)$. La differenza grossa sta nel fatto che in un approccio ci siamo salvati ogni volta le soluzioni, mentre nell'altro no. Per ottenere $\mathcal{O}(n \log n)$ anche con l'approccio ricorsivo dobbiamo ricorrere alla *memoization* ovvero salvarsi un vettore in cui salviamo tutte le soluzioni dei sottoproblemi.

§5 NP-completezza

L'NP-completezza studia se un problema è "difficile" o meno. Ovvero se per risolvere un determinato problema è necessario oppure no pagare un costo più che polinomiale (leggi esponenziale). Il focus qui più che sugli algoritmi è proprio sui problemi. Diamo una definizione informale

Definizione 5.1 (Easy Problem). Diciamo che un problema è *easy* se esiste un algoritmo che lo risolve con tempo di esecuzione $\mathcal{O}(n^c)$ con $c \in \mathbb{R}^+$.

Definizione 5.2 (Hard Problem). Diciamo che un problema è *hard* se non esiste (o non è ancora stato trovato) un algoritmo che ha running time al più $\mathcal{O}(n^c)$ per qualche $c \in \mathbb{R}^+$.

Vediamo subito un problema per fare un esempio:

Esempio 5.3 (Maximum Independent Set - MIS)

Definiamo il seguente problema computazionale:

Input Un grafo $G = (V, E)$ con n nodi e m archi.

Output Il massimo *insieme indipendente* di G . Un *insieme indipendente* in un grafo è un sottoinsieme di V in cui comunque scelti due nodi u e v non esiste un arco da u a v .

Questo è un problema hard. Notiamo che questo chiede di trovare il *maximum* independent set, esiste una variante che invece chiede di trovare il *maximal* independent set, che chiede di trovare un insieme indipendente che non si possa ulteriormente allargare; questo invece è un problema easy e si può risolvere con un approccio greedy.

In effetti questo è uno dei principali problemi di riferimento quando si parla di problemi hard, appartiene (insieme ad altri che vedremo poi) a una famiglia di problemi detti "NP-completi".

Esempio 5.4 (Minimum Vertex Cover)

Definiamo il seguente problema computazionale

Input Un grafo $G = (V, E)$ con n nodi e m archi.

Output Il minimo sottoinsieme S di V tale che ogni arco è adiacente ad almeno un nodo di S .

Problema 5.5. Questi due problemi sono relazionati in qualche modo?

Questa è esattamente la strategia che seguiremo per approcciare i problemi di NP-completezza, la *riduzione di un problema ad un altro*. Per i due problemi sopra non è ancora stata trovata alcuna soluzione polinomiale.

In effetti notiamo che

Lemma 5.6 (MIS=MVC)

Sia V un grafo. Date due soluzioni $\hat{s}$ e $\hat{p}$ rispettivamente per il MIS e per il MVC di V , allora vale sempre

$$\hat{s} = V \setminus \hat{p}$$

$$\hat{p} = V \setminus \hat{s}$$

Dimostrazione. Sia $\hat{s}$ una soluzione al MIS, allora è facile dimostrare che essendo esso massimale il suo complemento toccherà tutti gli archi esattamente una volta, dunque sarà un MVC. Analogamente si può fare per l'MVC. $\square$

quello che abbiamo appena fatto è *ridurre* MVC a MIS e viceversa. Ovvero abbiamo costruito un algoritmo che data la soluzione di uno dei due problemi trova la soluzione dell'altro. In particolare questo algoritmo va in tempo lineare (polinomiale), e quindi se risolvessimo uno dei due in tempo polinomiale, avremmo istantaneamente anche l'altro in tempo polinomiale.

Definizione 5.7 (problema riducibile in tempo polinomiale). Un problema Y è *riducibile in tempo polinomiale* ad X se ogni istanza di Y può essere risolta usando un numero polinomiale di operazioni generiche e un numero polinomiale di chiamate ad un algoritmo che risolve X . In questo caso scriviamo

$$Y \leq_p X.$$

In un certo senso quindi il problema Y è *più facile* di X .

Abbiamo appena visto che

$$\text{MIS} \leq_p \text{MVC}$$

e inoltre

$$\text{MVC} \leq_p \text{MIS}$$

dunque possiamo dire

$$\text{MIS} =_p \text{MVC}$$

Lemma 5.8 (Easier problems)

Siano X e Y due problemi computazionali tali che

$$Y \leq_p X$$

allora se esiste un algoritmo polinomiale per X esisterà anche per Y .

Dimostrazione. Per la definizione di riduzione possiamo scrivere

$$T_Y(n) = \mathcal{O}(n^c) + \mathcal{O}(n^c)T_X(k)$$

ora il punto di tutto è che k non può essere davvero più grande di n^c (il numero di operazioni già fatte) perché utilizzando operazioni elementari non possiamo far cambiare la taglia dell'istanza di T_Y a più di $n^c + n$. Dunque abbiamo:

$$T_Y(n) = \dots = \mathcal{O}(n^c) + \mathcal{O}(n^c)T_X(n^c) = \mathcal{O}(n^c) + \mathcal{O}(n^c)\mathcal{O}(n^{cl}) = \mathcal{O}(n^e)$$

che è polinomiale e quindi abbiamo finito. $\square$

Dunque applicando il converso di ciò abbiamo che se Y non può essere risolto in tempo polinomiale (e dimostriamo che è così) allora X non può essere risolto in tempo polinomiale.

§5.1 Decision Problems

Forti della teoria appena esposta ci interessa trovare una famiglia di problemi a cui possiamo ridurre la maggior parte dei problemi computazionali e di cui possiamo dire qualcosa sulla facilità.

I decision problems sono problemi a risposta sempre binaria, ovvero il cui unico output è "sì" o "no".

Esempio 5.9 (Decision per MIS - k -IS)

costruiamo il seguente problema riducibile a MIS.

Input: Un grafo $G = (V, E)$ con n nodi e m archi e un intero k .

Output: Dire se esiste un IS di taglia k .

questi saranno il tipo di problemi che vorremmo trattare.

Esempio 5.10 (Satisfiability problems/SAT)

Definiamo il seguente problema computazionale:

Input: Sia t_i una lista di variabili booleane. Definiamo la seguente *clause*

$$c_i = t_{i_1} \vee t_{i_2} \vee t_{i_3} \vee \dots \vee t_{i_l}$$

in cui potenzialmente appaiono anche delle *negazioni* dei singoli t_i . con una certa riindicizzazione di t_i e una certa indicizzazione degli c_i . A questo punto definiamo una *conjunctive normal form* (CNF) come

$$c = c_1 \wedge c_2 \wedge c_3 \wedge \dots \wedge c_k$$

per qualche k . È data una CNF.

Output: Vogliamo verificare se esiste ("sì"/"no") un'assegnazione di valori booleani (vero o falso) alle variabili t_i tale che la CNF sia vera.

Esempio 5.11 (3-SAT)

Definiamo il seguente problema computazionale

Input: Data una lista di variabili booleane x_i e una CNF del tipo

$$c = c_1 \wedge c_2 \wedge c_3 \wedge \dots \wedge c_k$$

dove ogni clause ha esattamente 3 termini.

Output: "Sì" se esiste un'assegnazione di valori tali che c è vera, "No" altrimenti.

il 3-SAT è uno dei primi problemi di questo tipo ad essere stato studiato. Chiaramente se 3-SAT è difficile anche SAT sarà difficile, perché è un caso particolare. Il punto di

questi tre problemi è che:

Lemma 5.12 (Il k -IS è difficile)

3-SAT è riducibile in tempo polinomiale a independent set

$$3\text{-SAT} \leq_p k\text{-IS}$$

Dimostrazione. Dobbiamo dimostrare che ogni clausola di un tri sat può essere ridotta in qualche modo ad un'istanza di un problema di grafi. Il trucco è ridurre ogni 3-clause ad un triangolo su un grafo.

$$c = (x_1 \vee x_4 \vee x_5) \wedge (\neg x_3 \vee \neg x_1 \vee x_4) \wedge \dots$$

dunque esisterà un triangolo tra i nodi x_1, x_4, x_5 uno tra i nodi $\neg x_3, \neg x_1$ e $x_4 \dots$. La cosa qui è che usiamo *gli stessi nodi* per variabili uguali, e in questo modo stiamo traducendo su un grafo le condizioni logiche che abbiamo. Ok, abbiamo risolto gli $\vee$, per risolvere i $\neg$ ci basta collegare con un arco i nodi x_i e $\neg x_i$ per ogni i . Vogliamo quindi trovare un k -IS di taglia esattamente uguale al numero di clause, se lo avessimo infatti per ogni elemento del k -IS potremmo associare 1 alla proposizione associata e 0 a tutte le altre. Dato che ci sono al più tanti triangoli quante clause, abbiamo che sicuramente per ogni triangolo ci sarà almeno un nodo dell'insieme, e quindi avremo c vera.

In generale quindi dato un CNF con k clausole ci riduciamo a voler trovare un k -IS con k nodi. Abbiamo fatto ciò in tempo polinomiale, e dunque abbiamo finito. $\square$

dato che in 60 anni di ricerca non è ancora stato scoperto un algoritmo polinomiale per il Tri-SAT è ragionevole pensare che k -IS sia un problema difficile.

§5.2 Una gerarchia di difficoltà

La relazione di riducibilità definisce un vero e proprio *ordine parziale* sull'insieme dei problemi computazionali. In particolare l'insieme dei problemi computazionali ha un sottoinsieme detto dei *problemi computabili*, ovvero l'insieme di tutti i problemi risolvibili con un algoritmo. In particolare qui dentro abbiamo due sottoinsiemi interessanti:

Definizione 5.13 (P vs NP). Il sottoinsieme dei problemi decisionali "facili" (ovvero, che ammettono una soluzione polinomiale) viene detto P . Sia ora Π un problema computazionale decisionale, diciamo Π appartiene all'insieme NP dei problemi "facili da verificare" se soddisfa la seguente condizione: esiste un $\beta(s, t)$ polinomiale (un algoritmo certificatore) che soddisfa: $\Pi(s) = \text{yes}$ se e solo se esiste un t di taglia polinomiale rispetto a s (un "certificato", o come dire, una soluzione, nel caso degli independent set è l'insieme dei nodi indipendenti) tale che $\beta(s, t) = \text{yes}$.

Possiamo immediatamente dire che $P \subseteq NP$, perché come algoritmo β "certificatore" ci basta utilizzare brutalmente l'algoritmo che risolve Π , in quanto è polinomiale. In effetti con la nostra teoria della riduzione possiamo fare di meglio, ovvero

Definizione 5.14 (problemi NP -completi). Un problema tale che ogni elemento di NP può essere ridotto ad esso si dice NP -completo.

Definizione 5.15 (Circuito). Un circuito è un grafo diretto aciclico con tre tipi di nodi:

Input Nodes: nodi senza archi in ingresso, che possono assumere solo valori binari in $\{0, 1\}$.

Output Nodes: nodi senza archi in uscita e che possono assumere solo valori binari in $\{0, 1\}$.

Internal Nodes : nodi con un'etichetta corrispondente ad un'operazione booleana (con un numero di input che abbia senso rispetto al numero di archi in entrata). In realtà si può dimostrare che qualunque operazione binaria può essere espressa usando solo $(\wedge, \vee \text{ e } \neg)$.

Insomma un circuito è un vero e proprio computer.

Esempio 5.16 (Circuit Satisfiability/CS)

Definiamo il seguente problema computazionale decisionale:

Input: Un *circuito* c .

Output: "Yes" se e solo se esiste un input per c che dia almeno un output "Yes".

questo problema è estremamente importante perché

Lemma 5.17 (CS è NP -completo)

Per ogni problema $\Pi \in NP$ vale

$$\Pi \leq_p CS$$

Dimostrazione. A partire dall'algoritmo $\beta(s, t)$ di check e dalle sue istanze s e t possiamo costruire un circuito c di dimensione polinomiale in s e t .¹ Rappresentando s e t come stringhe binarie, possiamo costruire un circuito che prende in input le due stringhe s e t e restituisce "yes" oppure "no".

Ora se noi fissiamo i valori di s come sono dati in input al problema Π abbiamo effettivamente un problema di CS, con input i valori di t e output l'output di $\beta(s, t)$.

Eseguiamo quindi CS, otterremo una stringa t e un output di $\beta(s, t)$. Ma per definizione di β questo è anche l'output di Π , e dunque abbiamo risolto Π . $\square$

Questo è stato il primo problema NP -completo a essere scoperto. Per definizione di NP -completezza non è difficile vedere che tutti i problemi NP -completi sono equivalenti. Ci piacerebbe molto trovare altri problemi NP -completi, a questo fine esiste una "procedura" detta **riduzione di Karp**.

Dato un problema Π per verificare se è NP -completo facciamo:

- (1) Verifichiamo che il problema è NP .
- (2) Selezioniamo un problema NP completo Y .
- (3) Dimostriamo che per ogni istanza s_Π di Π possiamo costruire un input s_Y per Y tale che

$$\Pi(s_\Pi) = \text{yes} \iff Y(s_Y) = \text{yes}.$$

chiaramente la chiamata a Π non deve essere necessariamente unica per la definizione di riduzione (basta che sia polinomiale), ma in ogni caso la maggior parte delle riduzioni tra problemi NP -completi di solito richiedono solo una chiamata.

¹questo è abbastanza informale, ma non è difficile convincersi che sia vero se si considera che ogni circuito è essenzialmente una rappresentazione scheletrizzata di un computer.

Sicuramente possiamo iniziare dicendo che 3-SAT è *NP*-completo, in quanto è solo un caso specifico di CS.

Esempio 5.18 (Subset Sum)

Definiamo il seguente problema computazionale decisionale:

Input: Un insieme s di numeri naturali e un numero target $t \in \mathbb{N}$.

Output: "yes" se e solo se esiste un sottoinsieme di s la cui somma degli elementi è esattamente t .

verifichiamo che questo problema è *NP*-completo:

- (1) Il problema è abbastanza chiaramente *NP*, il certificato è il sottoinsieme scelto che somma a t .

Il passo due è quello più difficile e bisogna affidarsi ad euristiche. Salta fuori che il problema di cui abbiamo bisogno è 3-Sat. Sia n la cardinalità di s , e k il numero di clause che stiamo usando. Costruiamo $2(n+k)$ numeri, ciascuno con $n+k$ cifre nel modo (abbastanza folle) seguente (facciamo un esempio per $n=3$ e $k=4$). Consideriamo la CNF seguente:

$$\begin{aligned} c = & (x_1 \vee \neg x_2 \vee x_3) \wedge \\ & (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge \\ & (\neg x_1 \vee \neg x_2 \vee x_3) \wedge \\ & (x_1 \vee x_2 \vee x_3) \end{aligned}$$

costruiamo i $2n$ numeri $v_1, \dots, v_{2n}$, in rappresentazione decimale, non binaria, nel seguente modo:

v_i	x_1	x_2	x_3	c_1	c_2	c_3	c_4	vincolo
$v_1 =$	1	0	0	1	0	0	1	$x_1 = 1$
$v_2 =$	1	0	0	0	1	1	0	$x_1 = 0$
$v_3 =$	0	1	0	0	0	0	1	$x_2 = 1$
$v_4 =$	0	1	0	1	1	1	0	$x_2 = 0$
$v_5 =$	0	0	1	0	0	1	1	$x_3 = 1$
$v_6 =$	0	0	1	1	1	0	0	$x_3 = 0$

dove alla riga i colonna c_j è stato messo un 1 se e solo se, assumendo **tutti gli altri elementi della clause falsi** e il vincolo della colonna più a destra vero, la clause è vera. Inoltre costruiamo i $2k$ numeri $s_1, s_2 \dots s_k$ e $s'_1, s'_2, \dots, s'_k$ come:

$s_1 =$	0	0	0	1	0	0	0
$s'_1 =$	0	0	0	2	0	0	0
$s_2 =$	0	0	0	0	1	0	0
$s'_2 =$	0	0	0	0	2	0	0
$s_3 =$	0	0	0	0	0	1	0
$s'_3 =$	0	0	0	0	0	2	0
$s_4 =$	0	0	0	0	0	0	1
$s'_4 =$	0	0	0	0	0	0	2

A questo punto applichiamo Subset Sum a questi $2(n+k)$ numeri, usando come target

$$t = 1114444$$

questo risolve automaticamente 3-SAT. Infatti la soluzione per subset-sum sarà costituita esattamente da numeri i cui vincoli corrispondono ad un assignment delle variabili x_i che soddisfa 3-SAT. Questa cosa si generalizza facilmente usando basi e target abbastanza grande. Dunque abbiamo dimostrato che

$$3\text{-SAT} \leq_p \text{S-Sum}.$$

Infine vediamo

Esempio 5.19 (Knapsack Problem)

Definiamo il seguente problema computazionale:

Input: Una lista di n elementi, ognuno con un valore v_i e una dimensione s_i , un limite di capacità B e un target K .

Output: "yes" se e solo se esiste un sottoinsieme degli oggetti forniti tali che la somma dei loro s_i è minore o uguale B e la somma dei loro v_i è maggiore o uguale a K .

questo è un problema NP -completo, e non è difficile vedere perché facendo una riduzione di Karp con subset sum.

§6 Approximate and Randomized algorithms

Abbiamo visto con lo studio della *NP*-completezza che alcuni algoritmi hanno costi *worst case* difficilmente aggirabili. Ma nel mondo reale non siamo obbligati ad attenerci al caso peggiore...qui nasce l'idea di "barare" attraverso degli algoritmi approssimati. In questi algoritmi "ci accontentiamo" di avere una soluzione sufficientemente vicina all'output che stiamo cercando, invece che esattamente quella desiderata.

Questo è ragionevole anche perché di solito nel mondo reale gli input hanno comunque del rumore, e quindi non ha troppo senso cercare una soluzione esatta se sarà comunque sporcata dagli errori degli input.

Inoltre i nostri algoritmi finora sono sempre stati *deterministici*, cioè a parità di input ci aspettiamo parità di output. Se tuttavia rilassiamo questo vincolo in modo ragionevole (per esempio, se costruissimo un algoritmo che 1 volta su 10000000 desse un output completamente fuori di testa) possiamo ottenere dei notevoli miglioramenti al costo del nostro algoritmo. Anche qui la cosa ha senso perché se la probabilità che l'algoritmo fallisca è sufficientemente bassa, allora è più probabile che ci sia un problema di hardware (o di altro tipo) che rende impossibile ottenere la soluzione giusta. Dunque in questa sezione proveremo a rilassare tre vincoli:

- Non consideriamo più necessariamente il worst case ("beyond worst case").
- Non vogliamo più necessariamente soluzioni esatte.
- Non vogliamo più necessariamente che l'algoritmo sia deterministico, sia rispetto alla qualità dell'output (ovvero se è giusto o è sbagliato), sia rispetto alla complessità (ovvero può accadere che a parità di output l'algoritmo restituisca l'output in complessità diverse).

La più famosa applicazione di ciò sono forse gli LLM, che di solito usano delle "primitive" costituite da algoritmi randomizzati.

§6.1 Approximate Algorithms

Definizione 6.1 (Approximation Ratio). Un algoritmo A per un problema X ha un *approximation ratio* $\rho(n)$ se per ogni input x di dimensione n il costo C (quando il problema ammette un qualche tipo di "costo" che è ragionevole rilassare) della soluzione trovata da A è al più differente di un fattore $f(n)$ dal costo ottimale C^* . Ovvero

$$\max \left\{ \frac{C}{C^*}, \frac{C^*}{C} \right\} \leq \rho(n)$$

In questo caso diciamo che A è $\rho(n)$ -approssimato.

In generale noi vorremmo che ρ sia una funzione "piccola", costante se possibile. La vera potenza di un algoritmo randomizzato però è quando la funzione è della forma

$$\rho(n) = 1 + \varepsilon$$

ovvero possiamo liberamente variare il tasso di approssimazione in base ai nostri bisogni variando contemporaneamente la complessità dell'algoritmo.

Esercizio 6.2. Trovare un algoritmo 2-approssimato per il k -IS.

Interessantemente questo esercizio è equivalente a dimostrare $P = NP$! Quando facciamo algoritmi approssimati invece, MVC è molto più facile di k -IS, infatti possiamo scrivere un algoritmo approssimato per il MVC nel seguente modo:

Algoritmo 6.3 (Maximum Vertex Cover) — Consideriamo il seguente algoritmo:

```

1: function APPROXVC( $G : (V, E)$ )
2:    $C := \emptyset$ 
3:   while  $E \neq \emptyset$  do
4:     Let  $(n, v)$  be an edge in  $E$ 
5:      $C \leftarrow C \cup \{n, v\}$ 
6:     Remove from  $E$  all edges adjacent to  $u, v$ 
7:   end while
8:   return  $C$ 
9: end function

```

La complessità di questo algoritmo risulta $\mathcal{O}(n + m)$. Dimostriamo ora

Lemma 6.4 (Correctness)

L'algoritmo sopra produce un vertex cover di taglia C tale che

$$|C^*| \leq |C| \leq 2|C^*|$$

dove C^* è il vertex cover minimo.

Dimostrazione. Uno dei due bound è banale, in quanto $|C^*|$ è definito come il minimo. Sia A il set degli archi di G che selezioniamo alla riga 4. Sicuramente tutti gli elementi di A non hanno vertici in comune fra loro, per come lo abbiamo generato. Il vertex cover generato dunque avrà una dimensione:

$$|C| = 2|A|$$

Ora sicuramente noi sappiamo che un insieme ottimale C^* copre tutti gli archi, in particolare dato che ogni arco di A è disgiunto dagli altri, in C^* ci deve essere un nodo per ogni elemento di A , dunque abbiamo

$$|C^*| \geq |A|$$

da cui abbiamo immediatamente la tesi. $\square$

quindi l'algoritmo che abbiamo fatto è 2-approssimato. Questo è un primo approccio, abbastanza rozzo, e si può facilmente migliorare (per esempio scegliendo i nodi con degree più alto). Il problema è che più aggiungiamo euristiche, più si possono generare worst case.

Vogliamo costruire una $1 + \varepsilon$ -approximation. La complessità di un tale algoritmo dipenderà da due variabili, quindi sarà $T(n, \varepsilon)$.

Definizione 6.5 (PTAS/FPTAS). Un algoritmo A $1 + \varepsilon$ -approssimato può essere di due tipi:

PTAS (Polynomial Time Approximation Scheme) Un algoritmo che per ogni input, fissato ε l'algoritmo è polinomiale in ε . Alcuni esempi sono

$$\mathcal{O}\left(\frac{n^2}{\varepsilon}\right)$$

$$\mathcal{O}\left(n^{1/\varepsilon}\right)$$

FPTAS (Fully Polynomial Time Approximation Scheme) Un algoritmo in cui la complessità è polinomiale sia in n che in $1/\varepsilon$.

Generalmente gli algoritmi non in P non possono essere PTAS e non FPTAS, perché facendo tendere ε sufficientemente a 0 (in modo che l'intervallo di affidabilità dell'algoritmo sia minore dello scatto discreto tra due possibili output) avremmo un algoritmo polinomiale per un problema non P .

§6.1.1 Knapsack Problem

Vogliamo trovare un FPTAS per il knapsack problem. Utilizziamo una DP, costruiamo n liste A_j tali che la lista A_j contiene la lista di tutte le possibili soluzioni usando solo i primi j elementi a nostra disposizione per cui saranno liste del tipo

$$A_j = \{(t_1, w_1), (t_2, w_2), \dots, (t_j, w_j)\}$$

dove i t_i sono la somma dei valori della soluzione, e w_i è la somma dei pesi. Ora sicuramente sappiamo che se nella lista A_j ci sono due soluzioni con $t_a \leq t_b$ e $w_a \geq w_b$ ci converrà sicuramente sempre la soluzione con indice b . Rimuoviamo allora dagli A_j tutte le coppie "dominate" ovvero tali che esiste un'altra coppia con costo minore e valore maggiore (se è tutto uguale ne scegliamo solo una delle due).

Sortiamo ora per i valori di t_i . Essendo che non ci sono coppie dominate, se cresce il valore deve crescere anche il costo. In questo modo abbiamo garantito che

$$|A_j| \leq B + 1$$

perché ogni peso può essere assunto al più una volta. Ma facendo lo stesso ragionamento sui t_i invece che sui w_i troviamo anche

$$|A_j| \leq V + 1$$

dove V è la somma di tutti i valori di tutti gli oggetti a nostra disposizione, dunque

$$|A_j| \leq \min\{B, V\} + 1.$$

Usiamo ora la DP vera e propria, il caso base è A_1 , che è semplicemente

$$A_1 = \{(0, 0), (v_1, s_1)\}$$

ora usiamo il famoso "metti o toglì", cioè

- 1: $A_{j+1} = A_j$
- 2: **for all** $(t, w) \in A_j$ **do**
- 3: $(t, w) \leftarrow (t + v_{j+1}, w + s_{j+1})$ only if $t + s_{j+1} \leq B$.
- 4: **end for**
- 5: Remove all dominated pairs

Dunque l'algoritmo completo è

Algoritmo 6.6 — Consideriamo il seguente algoritmo

- 1: **function** EXACTKP(s_i, v_i)
- 2: $A_1 \leftarrow \{(0, 0), (v_1, s_1)\}$
- 3: **for** $j = 2$ to n **do**
- 4: $A_{j+1} = A_j$
- 5: **for all** $(t, w) \in A_j$ **do**
- 6: $(t, w) \leftarrow (t + v_{j+1}, w + s_{j+1})$ only if $t + s_{j+1} \leq B$.
- 7: **end for**

```

8:      Remove all dominated pairs
9:  end for
10: return  $\max_{(t,w) \in A_j} \{t\}$ 
11: end function

```

La complessità di questo algoritmo è quella necessaria a scorrere lungo la tabella degli A_j , ovvero $\mathcal{O}(n \min\{B, V\})$. Ehm, abbiamo appena dimostrato $P = NP$?

Il problema è che la NP completezza di un problema è definita sulla base del numero di bit necessario a rappresentare l'input (la famosa taglia), e B aumenta esponenzialmente all'aumentare della taglia di n , infatti se abbiamo n elementi, ciascuno rappresentato con k bit abbiamo che B nel worst case deve essere il **massimo numero rappresentabile con k bit**, che è proprio 2^k . Quindi il running time in funzione della taglia in realtà è

$$\mathcal{O}(2^k kn)$$

che è esponenziale. Un algoritmo che è polinomiale in funzione delle variabili di input ma esponenziale sulla taglia si dice *pseudopolynomial*. Questo algoritmo in effetti è molto utile se B è piccolo, ma la vera potenza è che può essere esteso ad un FPTAS. Infatti se scegliamo un fattore di scala μ e dividiamo tutti i valori v_i e s_i per μ , anche la loro somma verrà scalata di un fattore μ , quindi la nostra complessità migliora di un fattore $\frac{1}{\mu}$. Il problema è che in questo modo l'output alla fine sarà sbagliato di un fattore μ , ma ci basterà rimoltiplicare tutto per μ per correggere questo errore. D'altra parte dato che stiamo lavorando con interi dovremo troncare a un certo punto, e quindi introdurremo un errore.

Questo è un esempio di algoritmo approssimato che "ammorbidendo" un po' i constraints sulla precisione migliora di molto il running time. Dobbiamo tuttavia quantificare quest'errore. Se scegliamo il valore effettivamente a

$$\mu = \frac{\varepsilon \max\{v_i\}}{2n}$$

questo ci dà una complessità di

$$\mathcal{O}(nV') = \mathcal{O}\left(n^2 \frac{M}{\mu}\right).$$

Più formalmente vogliamo esprimere tutti i v_i, s_i come multipli di μ . Capiamo bene che questo non è sempre possibile in modo esatto, quindi invece di dividere esattamente facciamo le divisioni intere:

$$v'_i = \left\lfloor \frac{v_i}{\mu} \right\rfloor$$

è chiaro che poi applicheremo semplicemente ExactKP ai vettori $\{v'_i\}, \{s'_i\}$, che ci darà una soluzione **opt'**, sbagliata di un fattore μ , quindi moltiplicando di nuovo per μ avremo una soluzione corretta (eccetto che per errori di troncamento quando abbiamo fatto la divisione intera). Questo, chiamando M il massimo di tutto ci dà una complessità di

$$\mathcal{O}\left(\frac{n^2 M}{\mu}\right)$$

e la risposta sarà al più sbagliata di un fattore μn . Ora esprimiamo la complessità in funzione di ε il rapporto tra la soluzione ottimale e quella trovata. Abbiamo

$$n\mu \leq \varepsilon M \leq \varepsilon \text{OPT}$$

da cui abbiamo

Lemma 6.7 (Correctness)

L'algoritmo sopra è FPTAS $1 + \varepsilon$ approximate con running time

$$\mathcal{O}\left(\frac{n^3}{\varepsilon}\right)$$

Dimostrazione. Sia opt la soluzione ottimale del problema di partenza, vogliamo mostrare che c'è un certo ε tale che

$$\text{opt} \leq \text{opt}'(1 + \varepsilon)$$

sia S la soluzione ottima data dall'algoritmo approssimato, allora usando la definizione μ :

$$\text{opt}' = \sum_{i \in S} v_i \geq \sum_{i \in S} v'_i \mu = \mu \sum_{i \in S} v'_i$$

d'altra parte se O è l'insieme di indici della soluzione ottimale su v_i (che non possiamo calcolare, ma assumiamo di avere), dato che non abbiamo scalato i costi avremo che la soluzione trovata sui v'_i è sempre minore o uguale di:

$$\dots \geq \mu \sum_{i \in O} v'_i$$

ma d'altra parte ricordando che

$$\mu(v'_i + 1) \geq v_i \iff \mu v'_i \geq v_i - \mu$$

abbiamo

$$\dots \geq \sum_{i \in O} (v_i - \mu) \geq \text{opt} - n\mu$$

quindi scegliendo

$$\mu = \frac{\varepsilon M}{2n} \iff \mu n = \frac{\varepsilon M}{2}$$

se sostituiamo abbiamo

$$\text{opt}' \geq \text{opt} - \frac{\varepsilon M}{2}$$

e ricordando che

$$M \geq \sum_{i \in S} v_i = \text{opt}'$$

□

§6.2 Randomized Algorithms

Definizione 6.8 (Las Vegas e Monte Carlo Algorithms). Un algoritmo randomizzato può essere *Las Vegas* se la correttezza dell'output è deterministica, ma la complessità del running time no. In algoritmi del genere di solito si esamina il valor medio della complessità $\mathbb{E}[T(n)]$.

Un algoritmo si dice *Monte Carlo* se non fornisce necessariamente sempre la soluzione corretta. Di solito per studiare un algoritmo Monte Carlo si definisce il parametro δ tale che, se definiamo la variabile aleatoria

$$X = \begin{cases} 1 & \text{se la soluzione è corretta} \\ 0 & \text{altrimenti.} \end{cases}$$

δ soddisfa

$$P[X = 1] \geq 1 - \delta.$$

§6.2.1 Quicksort

L'esempio per eccellenza di algoritmo Las Vegas è quicksort, ovvero

Algoritmo 6.9 (Quicksort) — Definiamo il seguente algoritmo di ordinamento:

```

1: function QUICKSORT( $S$ )
2:   if  $|S| \leq 3$  then
3:     Bruteforce the sort on  $S$ 
4:     return sorted  $S$ 
5:   end if
6:   Create empty vector  $S^+, S^-$ 
7:   Random select  $a_i \in S$  uniformly at random, independently
8:   for all  $a_j \in S$  do
9:     If  $a_j < a_i$  then put  $a_j$  in  $S^-$ 
10:    If  $a_j > a_i$  then put  $a_j$  in  $S^+$ 
11:   end for
12:    $\hat{S}^+ \leftarrow \text{QUICKSORT}(S^+)$ 
13:    $\hat{S}^- \leftarrow \text{QUICKSORT}(S^-)$ 
14:   return  $\hat{S}^- \cup a_i \cup \hat{S}^+$ 
15: end function

```

Per analizzare la complessità temporale di questo algoritmo definiamo il concetto di "pivot interno": un pivot che si trova tra il primo e l'ultimo quarto dell'array (ovvero che sia nel secondo o terzo quarto).

Se mettiamo dentro un while la parte in cui l'algoritmo splitta randomicamente l'array con la condizione che il pivot sia tra il primo e l'ultimo quarto dell'array (ovvero, che sia un pivo interno), abbiamo un numero atteso di iterazioni 2. Studiamo questo algoritmo:

Teorema 6.10

Quicksort ha un tempo di esecuzione atteso uguale a $\mathcal{O}(n \log n)$.

Dimostrazione. Chiamiamo un sottoproblema di tipo j se la sua dimensione rispetto all' n di partenza è tra

$$n \left(\frac{3}{4}\right)^{j+1} < |S| < n \left(\frac{3}{4}\right)^j$$

allora se un sottoproblema S è di tipo j anche $-/S^+$ sono di tipo $j' > j$ (con l'algoritmo modificato con i pivot interni). Inoltre i sottoproblemi sono sempre tutti distinti, quindi il massimo numero di problemi di tipo j che possiamo avere in tutto nell'albero è

$$\frac{n}{n \left(\frac{3}{4}\right)^j} = \left(\frac{4}{3}\right)^j < \# \text{sottoproblemi di tipo } j < \frac{n}{n \left(\frac{3}{4}\right)^{j+1}} = \left(\frac{4}{3}\right)^{j+1}$$

quindi in particolare se sommiamo su ogni tipo di sottoproblema abbiamo

$$T(n) = \sum_{j=0}^{\log_{4/3} n} \sum_{S \text{ di tipo } j} \mathcal{O}(S)$$

ora ci interessa studiare il valore atteso di questa cosa, quindi

$$\mathbb{E} \left(\sum_{j=0}^{\log_{4/3} n} \sum_{S \text{ di tipo } j} \mathcal{O}(S) \right)$$

usando la linearità del valore atteso dunque

$$\sum_{j=0}^{\log_{4/3} n} \sum_{S \text{ di tipo } j} \mathbb{E}(\mathcal{O}(S))$$

ma allora usando la definizione di sottoproblemi di tipo j e la stima sul numero di sottoproblemi di tipo j trovata prima, troviamo che il valore atteso della complessità di S è senza dubbio

$$\sum_{j=0}^{\log_{4/3} n} \sum_{S \text{ di tipo } j} \left(\frac{4}{3}\right)^{j+1} = \sum_{j=0}^{\log_{4/3} n} \frac{4}{3} n = \mathcal{O}(n \log_{4/3} n)$$

che era quanto volevamo dimostrare. $\square$

§6.2.2 Min Cut

Definizione 6.11 (Cut). Se $S \subseteq V$ il suo *cut* è definito come l'insieme

$$\delta(S) = \{(u, v) : u \in S, v \in V \setminus S\}$$

Ovvero è l'insieme degli archi che vanno da S a fuori S .

Esempio 6.12 (Min Cut)

Definiamo il seguente problema computazionale.

Input: Un certo grafo G non diretto.

Output: Il minimal cut, ovvero tra tutte le scelte di S , il cut con dimensione minima:

$$\operatorname{argmin}_{S \subseteq V} \{\delta(S)\}$$

la prima cosa che notiamo è che con la nostra definizione di cut il grado minimo è un upper bound del cut:

$$\min_{S \subseteq V} \{\delta(S)\} \leq \min_{v \in V} \{\deg(v)\}$$

in effetti con l'algoritmo di Ford-Fulkerson questo problema è risolvibile in $\mathcal{O}(n^3 m^2)$ utilizzando tecniche di flusso. Vogliamo fare di meglio, e per questo vogliamo usare un algoritmo Monte Carlo: L'algoritmo di Karger.

Definizione 6.13 (Contraction). Sia G un grafo. Una sua *contrazione* è un grafo G' (non necessariamente semplice, ovvero ci possono essere archi ripetuti) in cui "due nodi sono schiacciati tra loro" (nodi non necessariamente connessi). Ovvero si rimuovono due nodi u, v e si inserisce un nodo w e lo si collega a tutti i nodi che erano collegati con u o v .

Per tenere traccia di quello che facciamo aggiungiamo una label a ogni nodo, ovvero definiamo una funzione l tale che inizialmente $l(v) = \{v\}$ e che se contraiamo i nodi u e v al nodo w la label diventi

$$l(w) = l(u) \cup l(v)$$

Algoritmo 6.14 (Karger) — Definiamo il seguente algoritmo:

```

1: function CONTRACT( $G = (V, E)$ ,  $e = (u, v) \in E$ )
2:   replace  $u, v$  with node  $w$ .
3:    $l(w) = l(u) \cup l(v)$ 
4:   for all  $(u, x) \in E$  do
5:     Add  $(w, x)$  to  $E$ 
6:     remove  $(u, x)$  from  $E$ 
7:   end for
8:   for all  $(v, x) \in E$  do
9:     Add  $(w, x)$  to  $E$ 
10:    remove  $(v, x)$  from  $E$ 
11:  end for
12:  return  $G$ 
13: end function
14: function KARGER( $G = (V, E)$ )
15:  while  $|V| < 2$  do
16:    random select  $e \in E$ ,  $e = (u, v)$ 
17:    CONTRACT( $G, (u, v)$ )
18:  end while
19:   $S \leftarrow l(a)$   ▷ a questo punto l'insieme  $V$  conterrà solo due elementi, ovvero
     $V = \{a, b\}$ 
20:  return  $|\delta(S)|$ 
21: end function

```

Questo algoritmo se facciamo una prima analisi va esattamente in $\mathcal{O}(nm)$. Ora il claim chiave è

Lemma 6.15 (Probabilità di perdere)

L'algoritmo di Karger ha una probabilità di circa

$$\frac{1}{\binom{n}{2}} = \frac{1}{\mathcal{O}(n^2)}$$

di generare un cut minimale.

Dimostrazione. Siano $e_1, \dots, e_m$ gli archi di G scelti dall'algoritmo all' i -esima iterazione del while. Sia S il sottoinsieme dei vertici che dà min cut. Definiamo infine G_n il grafo alla n -esima contrazione.

Claim 6.16 — Ogni nodo in G_i ha grado $\geq |\delta(S)|$.

questo è chiaro in quanto se valesse l'opposto potremmo generare un set con cut minore di $|\delta(S)|$. Sappiamo che in G_i ci sono $n - i$ nodi. Usando il claim possiamo trovare un

lower bound al numero di archi di G_i come

$$|E_i| \geq \frac{|\delta(S)| (n-i)}{2}$$

la probabilità che l'algoritmo non fallisca è uguale alla probabilità di non scegliere mai un arco di $|\delta(S)|$, ovvero

$$P[e_1, e_2, \dots, e_n \notin \delta(S)] = P[e_1 \in \delta(S)] \prod_{j=2}^{n-2} P[e_j \notin \delta(S) \mid e_1, \dots, e_{j-1} \notin \delta(S)]$$

ma ora per calcolare queste probabilità ci basta fare un conto sul numero degli archi, ovvero

$$\dots = \left(1 - \frac{|\delta(S)|}{|\delta(S)| n/2}\right) \prod_{j=2}^{n-2} \left(1 - \frac{|\delta(S)|}{(n-j+1) |\delta(S)|/2}\right)$$

quindi semplificando molto convenientemente i δ quello che ci resta è

$$P[e_1, e_2, \dots, e_n \notin \delta(S)] = \dots = \frac{n-2}{n} \frac{n-3}{n-1} \frac{n-4}{n-2} \dots$$

che si semplifica come volevamo a

$$P[e_1, e_2, \dots, e_n \notin \delta(S)] = \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

□

Con questo lemma è chiaro che se facciamo andare n^2 volte l'algoritmo di Karger avremo un valore atteso di 1 al min cut, ottenendo una complessità di $\mathcal{O}(n^3 m)$. Se vogliamo garanzie probabilistiche di non perdere maggiori ci basta comunque aggiungere un fattore $\log n$, e per ragioni mistiche di probabilità abbiamo una probabilità di perdere ancora minore.

In effetti per quanto abbiamo visto prima più contraiamo più rischiamo di perdere. Questo ci motiva un'ottimizzazione dell'algoritmo di Karger che consiste nel "fermarsi prima".

Algoritmo 6.17 (Karger-Steiner) — Il seguente algoritmo è un miglioramento dell'algoritmo di Karger

```

1: function IMPROVEDALG( $G$ )
2:   if  $|V| \leq 6$  then
3:     Brute force
4:     return solutions
5:   else
6:      $G_1 \leftarrow$  the graph  $G$  contracted up to  $G/\sqrt{2}$  vertices with Karger's.
7:      $G_2 \leftarrow$  the graph  $G$  contracted up to  $G/\sqrt{2}$  vertices with Karger's.
8:     return  $\min(\text{IMPROVEDALG}(G_1), \text{IMPROVEDALG}(G_2))$ 
9:   end if
10: end function
```

questo algoritmo ha una complessità ricorsiva del tipo

$$T(n) = \begin{cases} 2T\left(\frac{n}{\sqrt{2}}\right) + \mathcal{O}(n^2) & \text{se } n > 1 \\ \mathcal{O}(1) & \text{altrimenti.} \end{cases}$$

che si riduce a $\mathcal{O}(n^2 \log n)$.

§7 Direzioni di ricerca

Esempio 7.1 (r-NN problem)

Descriviamo il seguente problema computazionale

Input: Un insieme S di n punti in $\mathbb{R}^d$.

Output: Data una query $q \in \mathbb{R}^d$, returna un punto p di S tale che esista una distanza $d(p, q)$ tale che $d(p, q) \leq r$.

La soluzione banale è in $\mathcal{O}(dn)$, ma questo combinato con il fatto che è un problema a query fa velocemente esplodere tutto. Vogliamo quindi implementare una struttura dati che ci faccia fare delle query in tempo migliore. Turns out, che in d dimensioni questo si può fare in con un kd -tree in:

$$\mathcal{O}\left(dn^{1-\frac{1}{d}}\right)$$

ma capiamo bene che per $d \rightarrow \infty$ questa tende alla soluzione triviale. In effetti capire se esistono soluzioni "truly sublinear" è un importante problema aperto e una direzione di ricerca in algoritmica. La versione approssimata di questo problema è:

Esempio 7.2 (r-ANN problem)

Descriviamo il seguente problema computazionale

Input: Un insieme S di n punti in $\mathbb{R}^d$.

Output: Data una query $q \in \mathbb{R}^d$, returna un punto x di S tale che esista una distanza $d(x, q)$ tale che

$$d(x, q) \leq cr$$

dove c è un approximation factor, se esiste un punto $p \in S$ tale che $d(p, q) < r$.

questa definizione è estremamente contorta, ma essenzialmente la presenza del $<$ è funzionale al non ricondursi brutalmente al problema esatto. In pratica abbiamo due palle intorno al punto q , e la seconda è "flessibile" infatti se abbiamo punti nella corona circolare più esterna e nessun punto nella palla più interna il comportamento è indefinito, ovvero che il nostro algoritmo dia risultati assurdi in quel caso! Con questa approssimazione possiamo raggiungere una complessità di

$$\mathcal{O}(dn^{\frac{1}{c^2}}).$$

Con una soluzione monte carlo e approssimata che si basa sul locality sensitive hashing possiamo raggiungere una complessità di

$$\mathcal{O}\left(n^{\frac{\log p_1}{\log p_2}}\right)$$

dove p_1 e p_2 sono le probabilità scelte dall'hashing locale tali che

$$p_1 < P[\text{due punti con distanza} \leq r \text{ abbiano lo stesso hash}]$$

$$p_2 > P[\text{due punti con distanza} \geq cr \text{ abbiano lo stesso hash}]$$

che alla fine si riconduce alla complessità con il c^2 .